```
> ######################################################################
  # Feynman_ploturi_structurate_foton_monopol.mpl
  #
  # Maple transcript -- STRUCTURED (grid) PLOTS for the two cases:
  #
  #   CASE 1 : virtual PHOTON  (standard QED, Born level)
  #   CASE 2 : magnetic monopole / "magnetron" -- CORRECTED variant
  #            (resonance selection rule + K-matrix unitarization)
  #
  # Each case is assembled as a labelled 1x3 panel grid (vertex diagram,
  # scattering diagram, observable/coupling panel), then both grids are
  # combined into a single 2x3 structured dashboard.
  #
  # Companion documents:
  #   Feynman_foton_si_monopol_corectat_RO.md /
  Feynman_photon_and_corrected_monopole_EN.md
  #       -- the consolidated write-up this transcript illustrates.
  #   Feynman_diagrame_foton_monopol_RO.md / EN,
  Feynman_monopol_unitarizare_RO.md / EN
  #       -- full derivations.
  #   monopole_wave_equation_full.md
  #       -- TauTimp Bessel spectrum k_{r,n}=j_{0,n}/(2R), used for Case 2.
  #   Feynman_diagrame_monopoli.mpl, Feynman_doua_metode_foton_magnetron.mpl
  #       -- earlier, non-grid Maple transcripts (single diagrams / printf
  report).
  #
  # Author : Viorel Popescu -- TauTimp / Teoria Informationala a Quarcilor
  #
  # Usage  : read "Feynman_ploturi_structurate_foton_monopol.mpl";
  #          or execute section by section (### ... ### delimiters).
  ######################################################################

  restart:
  with(plots):
  with(plottools):

  ######################################################################
  ### SECTION 0 -- drawing primitives, shared by both cases
  ######################################################################

  FermionLine := proc(p0::list, p1::list, {col::string := "black"})
      local seg, mid, dv, arr;
      seg := plottools:-line(p0, p1, color=col, thickness=3);
      mid := [(p0[1]+p1[1])/2, (p0[2]+p1[2])/2];
      dv  := [p1[1]-p0[1], p1[2]-p0[2]];
      arr := plottools:-arrow(mid, [mid[1]+dv[1]/20, mid[2]+dv[2]/20],
                              0.05, 0.15, 0.08, color=col);
      return plots:-display(seg, arr);
```

```
end proc:

PhotonLine := proc(p0::list, p1::list, {n::posint := 6, amp::numeric :=
0.2,
                    col::string := "blue"})
    local L, ux, uy, px, py, X, Y;
    L  := sqrt((p1[1]-p0[1])^2 + (p1[2]-p0[2])^2);
    ux := (p1[1]-p0[1])/L; uy := (p1[2]-p0[2])/L;
    px := -uy; py := ux;
    X  := t -> p0[1] + ux*t + px*amp*sin(2*Pi*n*t/L);
    Y  := t -> p0[2] + uy*t + py*amp*sin(2*Pi*n*t/L);
    return plot([X(t), Y(t), t=0..L], color=col, thickness=3);
end proc:

MonopoleLine := proc(p0::list, p1::list, {n::posint := 8, amp::numeric :=
0.25,
                      col::string := "red", lw::numeric := 3.5})
    local L, ux, uy, px, py, ts, pts, jj;
    L  := sqrt((p1[1]-p0[1])^2 + (p1[2]-p0[2])^2);
    ux := (p1[1]-p0[1])/L; uy := (p1[2]-p0[2])/L;
    px := -uy; py := ux;
    ts := [seq(jj*L/(2*n), jj=0..2*n)];
    pts := [ [p0[1], p0[2]],
             seq( [p0[1]+ux*ts[jj+1]+px*amp*(-1)^jj, p0[2]+uy*ts[jj+1]+py*
amp*(-1)^jj], jj=1..2*n-1),
             [p1[1], p1[2]] ];
    return plot(pts, style=line, color=col, thickness=lw);
end proc:

ThickMonopoleLine := proc(p0::list, p1::list, {n::posint := 8,
amp::numeric := 0.30,
                           col::string := "red"})
    return MonopoleLine(p0, p1, n=n, amp=amp, col=col, lw=5.5);
end proc:

DiracStringLine := proc(p0::list, dv::list, {len::numeric := 2.2,
col::string := "gray"})
    local nrm, ux, uy, p1;
    nrm := sqrt(dv[1]^2+dv[2]^2);
    ux  := dv[1]/nrm; uy := dv[2]/nrm;
    p1  := [p0[1]+ux*len, p0[2]+uy*len];
    return plottools:-line(p0, p1, color=col, linestyle=dash, thickness=
1.5);
end proc:

VertexDot := proc(p::list, {r::numeric := 0.08, col::string := "black"})
    return plottools:-disk(p, r, color=col);
end proc:
```

```
Blob := proc(p::list, {r::numeric := 0.45, label::string := "K"})
    return plots:-display(
        plottools:-circle(p, r, color=black, thickness=2),
        plots:-textplot([p[1], p[2], label], font=[TIMES,BOLD,13])
    );
end proc:

txt := proc(p::list, s, {fs::numeric := 12, col::string := "black"})
    return plots:-textplot([p[1], p[2], s], font=[TIMES,BOLD,fs], color=
col, align=above);
end proc:

# A blank-axes "text panel" -- used to show formulas/numbers as a grid
cell,
# on equal footing with the diagram panels (same panel size, no visible
axes).
TextPanel := proc(lines::list, {ttl::string := ""})
    local n, ys, k, items;
    n := nops(lines);
    ys := [seq(9 - 8*(k-1)/max(n-1,1), k=1..n)];
    items := [seq( plots:-textplot([1, ys[k], lines[k]], align=right,
font=[TIMES,12]), k=1..n)];
    return plots:-display(op(items), view=[0..10,0..10], axes=none,
                          title=ttl, scaling=constrained);
end proc:

#########################################################################
### CASE 1 -- virtual PHOTON (standard QED, Born level)
#########################################################################

# --- Panel 1A: elementary vertex ------------------------------------
V := [5,5]:
panel1A := plots:-display(
    FermionLine([1,2], V), FermionLine(V, [1,8]),
    PhotonLine(V, [9,5], n=5, amp=0.22, col="blue"),
    VertexDot(V),
    txt([0.6,1.7],"e-"), txt([0.6,8.3],"e-"), txt([9.2,5.5],"gamma*", col=
"blue"),
    title="1A -- elementary QED vertex", axes=none, scaling=constrained
):

# --- Panel 1B: Moller scattering, t-channel photon exchange ----------
V1 := [2.5,5]: V2 := [8,5]:
panel1B := plots:-display(
    FermionLine([2.5,1], V1), FermionLine(V1, [2.5,9]),
    FermionLine([8,1],  V2), FermionLine(V2, [8,9]),
    PhotonLine(V1, V2, n=6, amp=0.24, col="blue"),
    VertexDot(V1), VertexDot(V2),
    txt([2.5,0.4],"p1"), txt([2.5,9.4],"p3"), txt([8,0.4],"p2"), txt([8,
```

```
9.4],"p4"),
    title="1B -- Moller scattering (t-channel)", axes=none, scaling=
constrained
):

# --- Panel 1C: coupling / observable summary -------------------------
alpha_val := evalf(1/137.036):
panel1C := TextPanel(
    [ "Method 1: virtual photon",
      cat("alpha = ", convert(alpha_val,string)),
      cat("alpha/Pi = ", convert(evalf(alpha_val/Pi),string)),
      "Born level = full answer",
      "(no resummation needed)",
      "status: confirmed (g-2, 1e-12)" ],
    ttl="1C -- observable summary"
):

Case1Grid := plots:-display(Matrix(1,3,[panel1A,panel1B,panel1C]),
                            title="CASE 1 -- virtual photon (QED, Born
level)"):
print(Case1Grid);

#########################################################################
### CASE 2 -- monopole / "magnetron", CORRECTED (resonance +
unitarization)
#########################################################################

# --- numeric backbone (self-contained; matches the companion documents)
e_val    := evalf(sqrt(4*Pi*alpha_val)):
n_min    := 1:
g_val    := evalf(2*Pi*n_min/e_val):
alpha_g := evalf(g_val^2/(4*Pi)):
R_cosmo := 3.9228e22:
j01      := evalf(BesselJZeros(0,1)):
kr1      := j01/(2*R_cosmo):
K0       := evalf(alpha_g):                  # rho=1, on resonance (worst
case)
absa2_0 := evalf(K0^2/(1+K0^2)):

# --- Panel 2A: elementary monopole vertex + Dirac string -------------
V := [5,5]:
panel2A := plots:-display(
    FermionLine([1,2], V), FermionLine(V, [1,8]),
    MonopoleLine(V, [9,5], n=6, amp=0.26, col="red"),
    DiracStringLine(V, [0.6,0.85], len=2.4),
    VertexDot(V),
    txt([0.6,1.7],"e-/p"), txt([0.6,8.3],"e-/p"), txt([9.2,5.5],"M(g)",
col="red"),
    title="2A -- monopole vertex + Dirac string", axes=none, scaling=
```

```
constrained
):

# --- Panel 2B: corrected diagram -- dressed exchange, resonant q only -
V1 := [2.5,5]: V2 := [8,5]: Vm := [5.25,5]:
panel2B := plots:-display(
    FermionLine([2.5,1], V1), FermionLine(V1, [2.5,9]),
    FermionLine([8,1],  V2), FermionLine(V2, [8,9]),
    ThickMonopoleLine(V1, V2, n=8, amp=0.30, col="red"),
    Blob(Vm),
    DiracStringLine(V1, [-0.6,0.75], len=2.0),
    DiracStringLine(V2, [ 0.6,0.75], len=2.0),
    VertexDot(V1), VertexDot(V2),
    txt([2.5,0.4],"p1"), txt([2.5,9.4],"p3"), txt([8,0.4],"p2"), txt([8,
9.4],"p4"),
    title="2B -- corrected: dressed exchange, resonant q", axes=none,
scaling=constrained
):

# --- Panel 2C: unitarity saturation curve, |a(K)|^2 = K^2/(1+K^2) -----
# genuine Maple function plot, with the physical point (K0, |a|^2) marked.
curve2C := plot(Kx^2/(1+Kx^2), Kx=0..60,
    color=red, thickness=2.5,
    labels=["K = alpha_g*rho", "|a|^2"],
    title="2C -- unitarity saturation"
):
markPoint := plots:-pointplot([[K0, absa2_0]], symbol=solidcircle,
symbolsize=16, color=navy):
guideLines := plots:-display(
    plottools:-line([K0,0],[K0,absa2_0], linestyle=dot, color=navy),
    plottools:-line([0,absa2_0],[K0,absa2_0], linestyle=dot, color=navy)
):
boundLine := plots:-display(plottools:-line([0,1],[60,1], linestyle=dash,
color=black)):
panel2C := plots:-display(curve2C, boundLine, guideLines, markPoint, view=
[0..60,0..1.08]):

Case2Grid := plots:-display(Matrix(1,3,[panel2A,panel2B,panel2C]),
                            title="CASE 2 -- monopole/magnetron,
corrected (resonance + K-matrix)"):
print(Case2Grid);

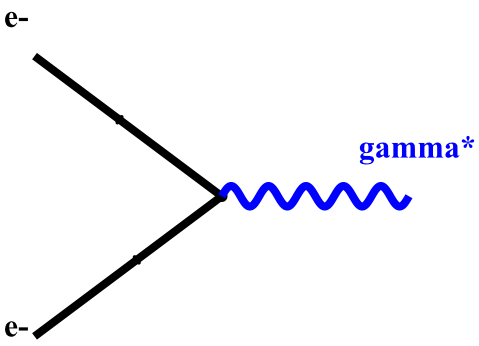
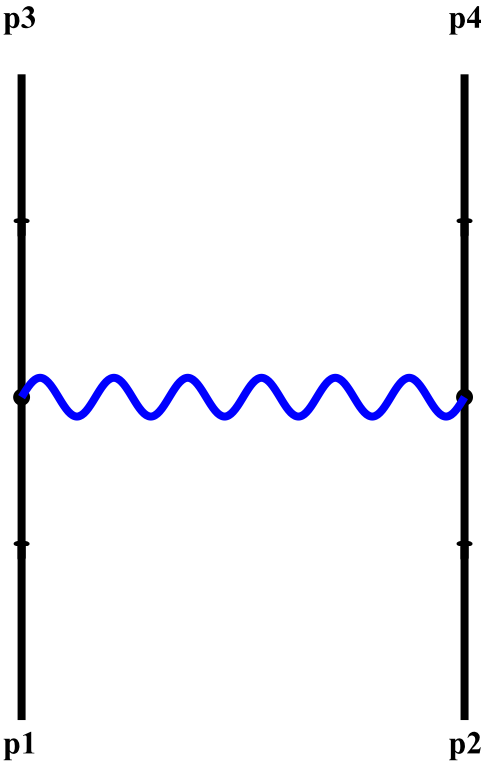
printf("Case 2 numeric backbone: alpha_g=%.4f, K0=%.4f, |a|^2=%.6f, |a|=
%.6f\n",
        alpha_g, K0, absa2_0, sqrt(absa2_0));

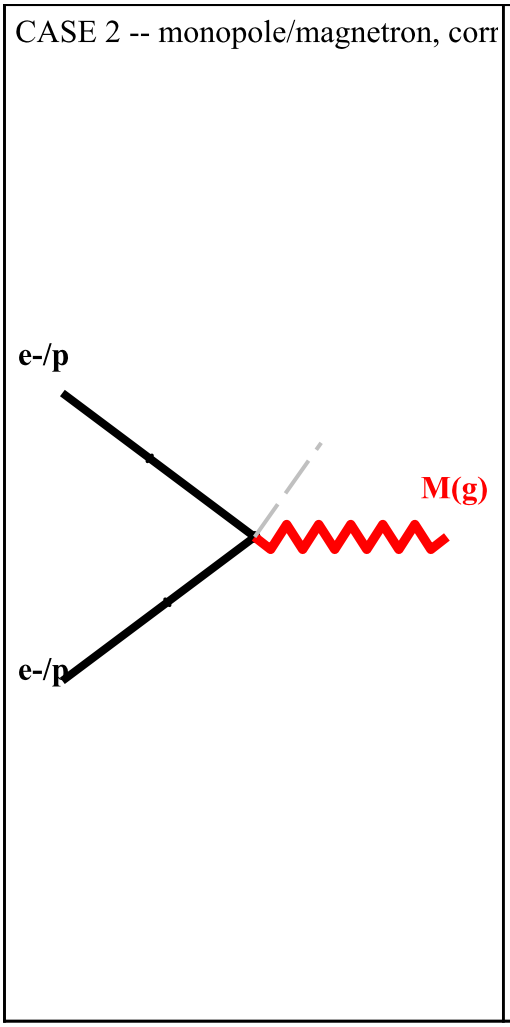
#########################################################################
### SECTION D -- combined structured dashboard (2 rows x 3 columns)
#########################################################################
```

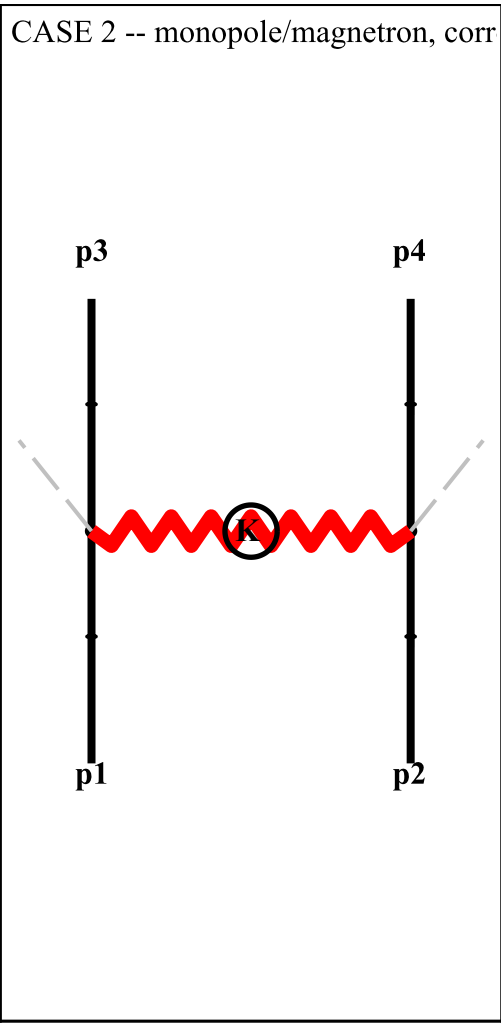
```
Dashboard := plots:-display(
    Matrix(2,3,[panel1A,panel1B,panel1C, panel2A,panel2B,panel2C]),
    title="Structured comparison: Case 1 (photon) vs. Case 2 (monopole,
corrected)"
):
print(Dashboard);

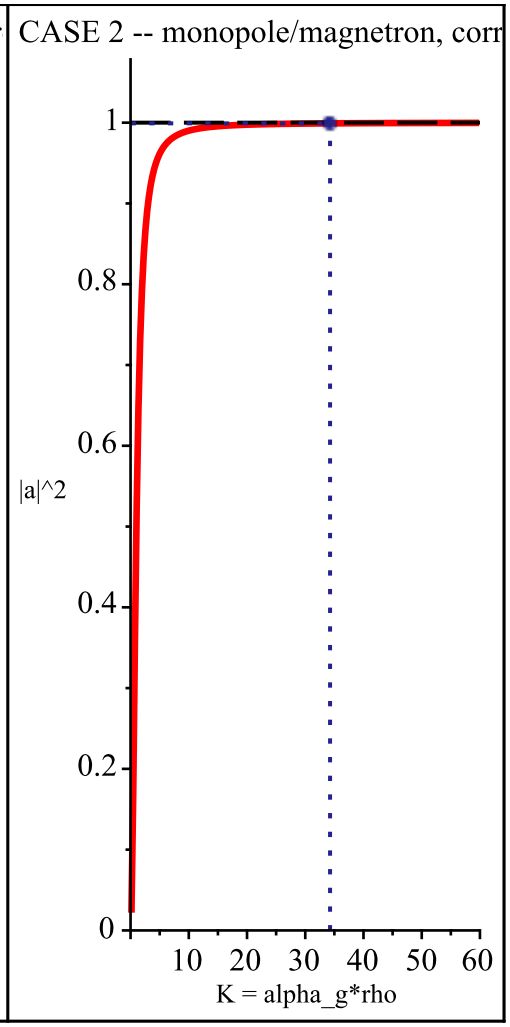
printf("\n--- End of transcript. Panels: 1A/1B/1C (photon), 2A/2B/2C
(monopole, corrected).\n");
printf("    See Feynman_foton_si_monopol_corectat_RO.md /
Feynman_photon_and_corrected_monopole_EN.md\n");
printf("    for the full write-up these structured plots illustrate.\n");

##########################################################################
# End of Feynman_ploturi_structurate_foton_monopol.mpl
##########################################################################
```

CASE 1 -- virtual photon (QED, Born	CASE 1 -- virtual photon (QED, Born	CASE 1 -- virtual photon (QED, Born
		Method 1: virtual photon alpha = .7297352520e-2 alpha/Pi = .2322819450e-2 Born level = full answer (no resummation needed) status: confirmed (g-2, 1e-12)

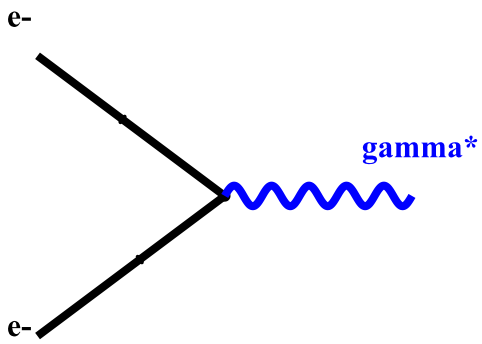




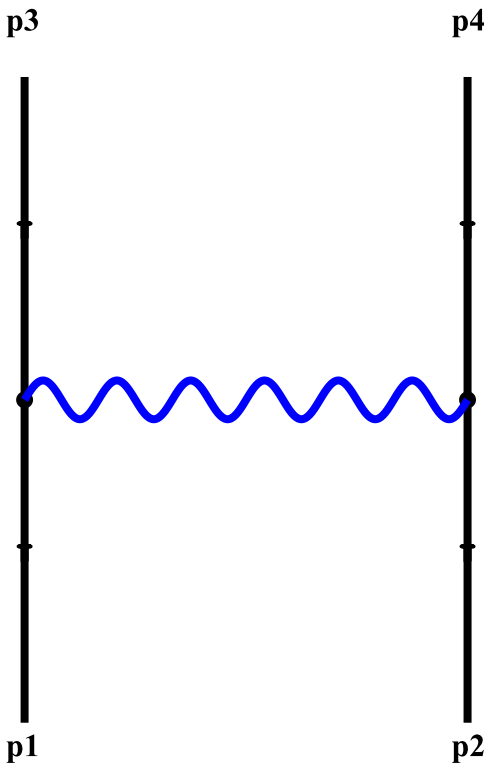


Case 2 numeric backbone: alpha_g=34.2590, K0=34.2590, |a|^2=0.999149, |a|= 0.999574

Structured comparison: Case 1 (photo



Structured comparison: Case 1 (photo



Structured comparison: Case 1 (photo

Method 1: virtual photon

alpha = .7297352520e-2

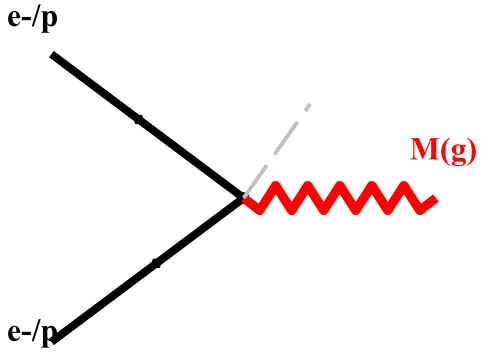
alpha/Pi = .2322819450e-2

Born level = full answer

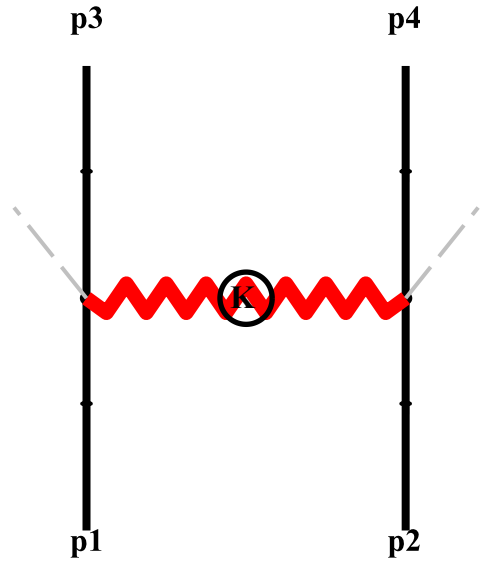
(no resummation needed)

status: confirmed (g-2, 1e-12)

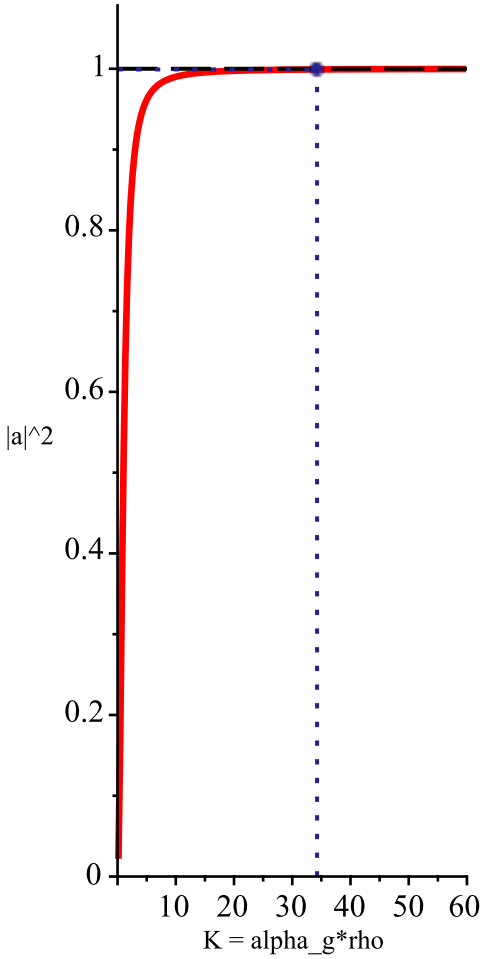
Structured comparison: Case 1 (photo



Structured comparison: Case 1 (photo



Structured comparison: Case 1 (photo



--- End of transcript. Panels: 1A/1B/1C (photon), 2A/2B/2C (monopole, corrected).
    See Feynman_foton_si_monopol_corectat_RO.md / Feynman_photon_and_corrected_monopole_EN.md
    for the full write-up these structured plots illustrate.